CV OBJECT DETECTOR & FRAME-BY-FRAME SEGMENTER PIPELINE

A Computer Vision Pipeline Using OpenCV and YOLO11n

Author: Asher Kashif **Project Type:** Computer Vision Mini Project

Technologies: Python, OpenCV, Ultralytics YOLO, NumPy, Pandas, Matplotlib

ABSTRACT

This project presents a frame-by-frame computer vision pipeline combining traditional OpenCV image processing with a pretrained YOLO11n object detection model. Each video frame is converted to grayscale, smoothed using a Gaussian filter, processed with adaptive thresholding, and analyzed for contours. In parallel, the original frame is passed to YOLO for object detection using bounding boxes, class labels, and confidence scores. Dynamic overlays display frame number, object count, contour count, and processing FPS. The experiment processed **1,786 frames**, with an average of **7.79 detected objects per frame**, a minimum of **2**, and a maximum of **16**. Average contour count was **229.13 per frame**. Average YOLO inference time was **0.2641 seconds**, average complete processing time was **0.3198 seconds**, and average processing speed was **3.16 FPS**.

1. INTRODUCTION

Computer vision enables software systems to extract useful information from images and video. Video processing requires repeated frame analysis while maintaining useful performance. Traditional techniques such as filtering, thresholding, and contour extraction provide low-level information about image structure, while deep-learning detectors such as YOLO provide semantic object information. This project combines both approaches. OpenCV is used for preprocessing and contour-based shape extraction, while YOLO11n is used for semantic object detection. Every video frame is processed independently, annotated, and measured.

2. PROJECT OBJECTIVES

- Process video frame-by-frame using Python and OpenCV.
- Apply Gaussian filtering and adaptive thresholding.
- Extract contours from thresholded frames.
- Detect objects using pretrained YOLO11n.
- Draw bounding boxes, labels, and confidence scores.
- Dynamically count detected objects.
- Record inference time, processing time, FPS, object count, and contour count.
- Generate CSV metrics and analytical visualizations.

3. SYSTEM ARCHITECTURE

Input Video → **Read Frame** → **Grayscale** → **Gaussian Filter** → **Adaptive Threshold** → **Contour Extraction**

Original Frame → **YOLO11n** → **Object Detection** → **Bounding Boxes + Classes + Confidence**

Both branches → **Dynamic Overlay** → **Metrics Recording** → **Output Video**

4. METHODOLOGY

4.1 Video Frame Extraction

OpenCV VideoCapture reads the input video sequentially. For every successful frame, the system records the frame number and starts a high-resolution timer. The experiment processed **1,786 frames**.

4.2 Grayscale Conversion

The original BGR frame is converted to grayscale for traditional image processing. This reduces the three-channel image to one intensity channel and simplifies filtering and thresholding. The original color frame remains available for YOLO detection.

4.3 Gaussian Filtering

A **5×5 Gaussian filter** is applied to the grayscale frame. Gaussian filtering reduces small image variations and noise before thresholding and contour extraction.

4.4 Adaptive Thresholding

Adaptive Gaussian thresholding is applied using block size **11** and constant **C=2**. Local threshold values make the method useful under varying illumination.

4.5 Contour Extraction

Contours are extracted using OpenCV findContours with external retrieval and chain approximation. Contour count is stored for every frame. Contours represent image regions and are not equivalent to semantic object instances.

5. YOLO OBJECT DETECTION

The deep-learning component uses the pretrained **YOLO11n** model through Ultralytics. The model is loaded using YOLO('yolo11n.pt'). For each frame, detections provide bounding-box coordinates, confidence scores, class IDs, and class names. The system draws each detection on the frame and displays its class and confidence. In this implementation YOLO performs object detection with bounding boxes, not instance segmentation.

5.1 Dynamic Object Counting

The number of YOLO detections is recalculated independently for every frame. The current count is displayed on the output video, providing frame-level object-count analysis.

5.2 Frame-Level Metrics

Metric	Description
Frame	Current frame number
Timestamp	Video time in seconds
Object count	Number of YOLO detections
Contour count	Number of extracted contours
Inference time	YOLO execution time
Processing time	Complete frame-processing time
Processing FPS	Frames processed per second

Metrics are saved to **results/frame_metrics.csv** and aggregate results to **results/summary_metrics.csv**.

6. EXPERIMENTAL SETUP

The project uses Python with OpenCV for computer vision, Ultralytics YOLO for detection, NumPy and Pandas for data processing, and Matplotlib for visualization. The dedicated project environment contains the required dependencies listed in requirements.txt.

6.1 Project Organization

input/ — source video
output/ — annotated output video
results/ — CSV metrics and summaries
src/ — pipeline and analysis scripts
visualizations/ — generated plots
whitepaper/ — documentation

7. EXPERIMENTAL RESULTS

The complete pipeline processed 1,786 video frames. The aggregate results are:

Metric	Actual Result
Total processed frames	1,786
Average objects per frame	7.79
Minimum objects per frame	2
Maximum objects per frame	16
Average contours per frame	229.13
Average YOLO inference time	0.2641 s
Average processing time	0.3198 s
Average processing FPS	3.16

7.1 Object Detection Statistics

The average YOLO object count was **7.79 objects per frame**. The minimum was 2 and the maximum was 16. Variation is expected because objects can enter or leave the camera view and their visibility changes over time.

7.2 Contour Statistics

The average contour count was **229.13 contours per frame**. This is substantially higher than the average YOLO object count because contours represent thresholded image regions rather than semantic objects. A single object may generate multiple contours, while background regions can also generate contours.

7.3 Inference Performance

Average YOLO inference time was **0.2641 seconds per frame**. The first frame required approximately **5.70 seconds**, treated as startup/model-initialization overhead rather than representative steady-state performance.

7.4 Overall Processing Performance

Average complete processing time was **0.3198 seconds per frame**, resulting in an average processing rate of **3.16 FPS**. Complete processing includes preprocessing, contour extraction, YOLO inference, annotation, metric collection, and output writing.

8. VISUALIZATION AND ANALYSIS

The analysis script generates four main graphs: object count over time, contour count over time, inference time over time, and processing FPS over time.

8.1 Object Count Over Time

This plot shows how the number of YOLO detections changes throughout the video. Increasing values indicate more detections in a frame, while decreasing values indicate fewer detections.

8.2 Contour Count Over Time

This plot shows the number of contours extracted by the OpenCV branch. Variation can be affected by thresholding, image texture, lighting, shadows, and background structures.

8.3 Inference Time Over Time

This graph records YOLO execution time for every frame. The first-frame spike reflects startup/model initialization overhead. Later values better represent steady-state inference behavior.

8.4 Processing FPS Over Time

This graph displays the processing rate derived from frame processing time. The experiment produced an average of 3.16 FPS.

9. DISCUSSION

The results demonstrate the complementary nature of traditional computer vision and deep-learning detection. OpenCV provides low-level image and shape information, while YOLO provides semantic object detections.

The difference between **229.13 average contours** and **7.79 average YOLO objects** confirms that these measurements represent different concepts. Contours should therefore not be interpreted as object instances.

The performance results show that repeated object detection is computationally significant. The average complete processing rate of 3.16 FPS indicates that the current implementation is primarily an experimental/offline processing pipeline rather than a high-frame-rate real-time system.

10. LIMITATIONS

- Average throughput is 3.16 FPS, below common real-time video rates.
- Objects are detected independently in each frame; persistent tracking IDs are not maintained.
- Contour extraction is traditional shape extraction, not semantic or instance segmentation.
- Image-processing parameters are fixed and may not be optimal for every scene.
- The pretrained YOLO model is not fine-tuned for a task-specific dataset.

11. FUTURE IMPROVEMENTS

- **Object Tracking:** Integrate a multi-object tracker such as ByteTrack to maintain persistent object identities.
- **GPU Acceleration:** Use compatible GPU hardware to reduce YOLO inference time and increase throughput.
- **Frame Skipping:** Combine periodic detection with tracking between detections to reduce computation.
- **True Instance Segmentation:** Use a segmentation-capable model when pixel-level object masks are required.
- **Parameter Optimization:** Experiment with Gaussian and threshold parameters for different environments.
- **Confidence Filtering:** Apply a configurable confidence threshold to reduce low-confidence detections.
- **Detailed Profiling:** Measure preprocessing, inference, annotation, and video-writing time separately.

12. CONCLUSION

A complete frame-by-frame computer vision pipeline was implemented using OpenCV and pretrained YOLO11n. The system successfully processed **1,786 frames**, generated dynamic object detections and overlays, extracted contour information, and recorded detailed performance metrics.

The experiment achieved an average of **7.79 detected objects per frame**, with a range of 2 to 16. The average contour count was **229.13 per frame**. Average YOLO inference time was **0.2641 seconds**, average complete processing time was **0.3198 seconds**, and average processing speed was **3.16 FPS**.

The project demonstrates a practical integration of traditional computer vision and deep-learning object detection. OpenCV provides preprocessing and shape information, while YOLO provides semantic object detection. Generated CSV files and visualizations make the experiment measurable and reproducible.

13. PROJECT OUTPUTS

results/frame_metrics.csv — per-frame measurements
results/summary_metrics.csv — aggregate metrics
output/detected_output.mp4 — annotated video
visualizations/ — generated analysis graphs
visualizations/sample_detection.jpg — sample annotated frame

14. REFERENCES

1. OpenCV Documentation — computer vision and image processing operations.
2. Ultralytics Documentation — YOLO object detection framework.
3. Python Documentation — Python programming language.
4. Pandas Documentation — data analysis and CSV processing.
5. Matplotlib Documentation — scientific data visualization.

Key Experimental Configuration

YOLO11n • Gaussian kernel 5×5 • Adaptive threshold block size 11 • C=2 • 1,786 frames • 7.79 average objects/frame • 229.13 average contours/frame • 0.2641 s average inference • 0.3198 s average processing • 3.16 FPS